

1. Introduction & OOP Design

Oct 2, 2026 · @Delya

1.1 Project Overview

The University Textbook Rental Hub lets students rent course textbooks. A student asks for a book by its ID. The system checks that the book exists and that a copy is available, then records the rental and reduces the stock.

Following the homework, we first wrote and tested this logic as plain Python classes, without any networking. The logic is split into two classes, `BookCatalog` and `RentalManager`, and three small data classes. All data is kept in memory, and the rules are deliberately simple.

1.2 OOP Class Design

BookCatalog is the inventory class. It stores books in a private dictionary that maps a book ID (for example `B001`) to a `Book` with a title, price and stock count. It knows nothing about students or rentals.

RentalManager is the business logic class. Its main method, `rent_book(student_id, book_id)`, rejects an empty student ID, then asks the catalog whether the book exists and whether a copy is available. If both checks pass, it reduces the stock, creates a `Rental` with a new ID (for example `R0001`) and stores it in a dictionary of active rentals.

Class	Main methods
<code>BookCatalog</code>	<code>add_book</code> , <code>get_book</code> , <code>list_books</code> , <code>search_books</code> , <code>check_stock</code> , <code>is_available</code> , <code>decrease_stock</code> , <code>increase_stock</code>
<code>RentalManager</code>	<code>rent_book</code> , <code>get_active_rentals</code> , <code>return_book</code>

Three dataclasses carry the data: `Book` (ID, title, price, stock), `Rental` (rental ID, student ID, book ID, price) and `RentalResult`. Every call to `rent_book` returns a `RentalResult` with `success`, `message`, `rental_id` and `remaining_stock`. A failed request, such as an unknown book or no copies left, is returned as `success = False` with a message rather than as an exception.

The classes follow a few basic OOP principles:

- **Single responsibility.** `BookCatalog` handles inventory and `RentalManager` handles the rental rules.
- **Encapsulation.** Stock changes only through `decrease_stock` and `increase_stock`, which do not let stock go below zero. The search methods return copies of books, so outside code cannot change the stored stock directly.
- **Composition.** `RentalManager` uses a `BookCatalog` object that is created outside and passed to its constructor. This lets each test use a fresh catalog, and lets several callers share the same one.

Two small features go beyond the homework requirements. `return_book` closes a rental and puts the copy back in stock. A `threading.Lock` in `RentalManager` stops two parallel requests from renting the same last copy.

1.3 UML Design

University Textbook Rental Hub - Class Diagram

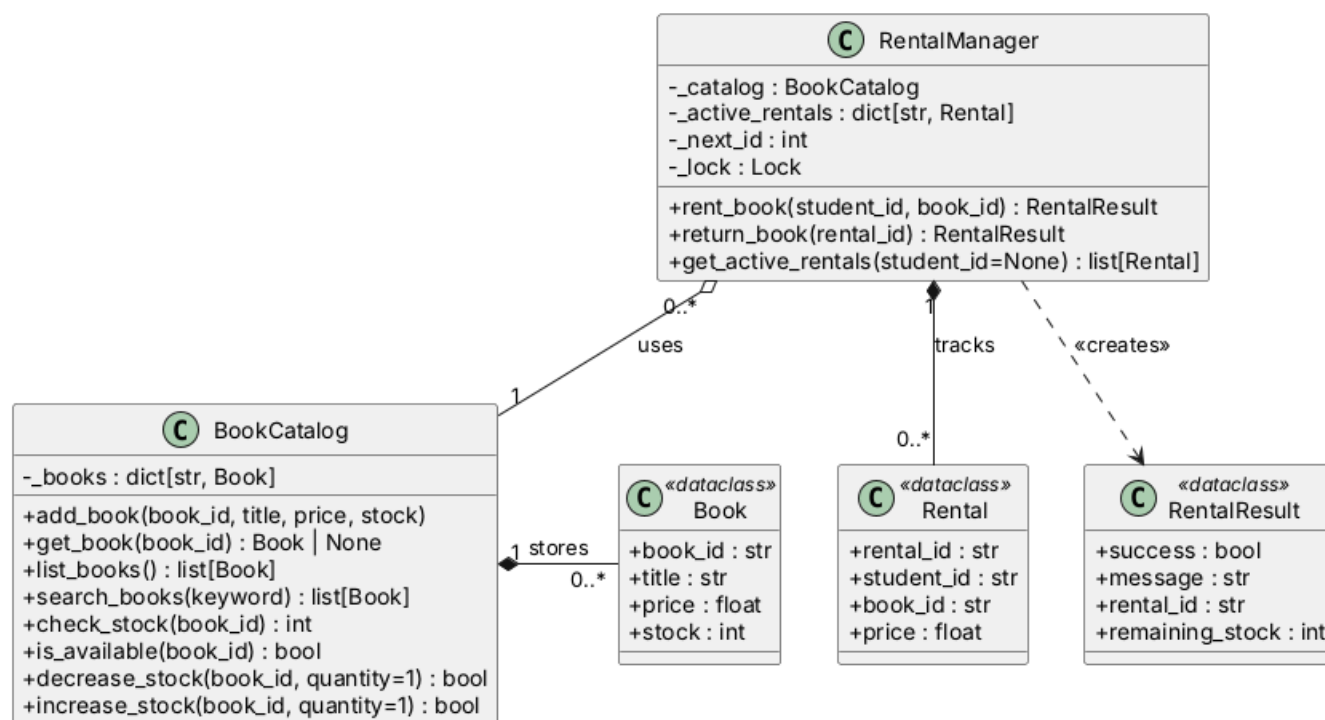


Figure 1. UML class diagram of the OOP layer.

The filled diamonds show composition: `BookCatalog` owns its `Book` objects, and `RentalManager` owns its `Rental` objects. The hollow diamond shows aggregation. `RentalManager` uses one `BookCatalog`, but the catalog is created outside and can exist without it. The dashed arrow means `RentalManager` creates a `RentalResult` for each request but does not store it. `Rental` keeps only a `book_id` string, so it has no direct link to `Book`.

1.4 Local Testing and gRPC Readiness

We tested the classes locally before writing any gRPC code. `demo.py` walks through a typical session with five sample books. `test_rental_hub.py` has 13 unit tests written with Python's `unittest` module. They cover adding and finding books, searching, checking stock, a successful rental, the stock reduction after it, and rental attempts for an unavailable book and a nonexistent book. They also check that active rentals are stored correctly. All 13 tests passed when we ran `python -m unittest -v` on Python 3.13.

The OOP classes do not contain any networking code. They take plain values such as IDs and keywords, and return plain objects. Because of this, Persons 2 and 3 can reuse them unchanged from their gRPC service handlers. A handler reads the request fields, calls `RentalManager` or `BookCatalog`, and copies the returned values into a response message:

```
gRPC request → service handler → RentalManager → BookCatalog → result → gRPC response
```

The `.proto` contract and the synchronous and asynchronous servers are described in the following sections.